



# Lecture # 26-28 Query Processing

**Rashmi Dutta Baruah**

**Department of Computer Science & Engineering**



**भारतीय प्रौद्योगिकी संस्थान गुवाहाटी**  
**Indian Institute of Technology Guwahati**

Guwahati - 781039, INDIA

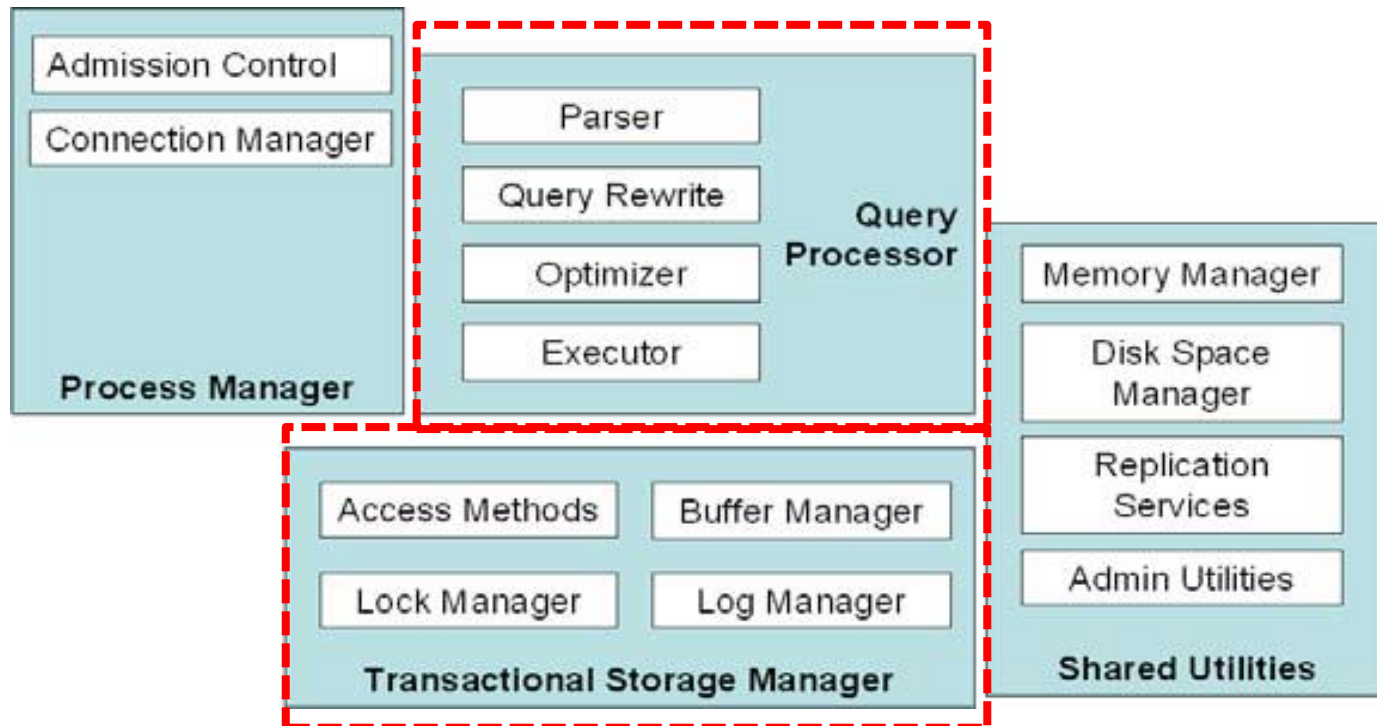




# Outline

- Overview
- Measures of Query cost
- Selection Operation
- Sorting
- Join Operation
- Evaluation of Expressions

# DBMS architecture



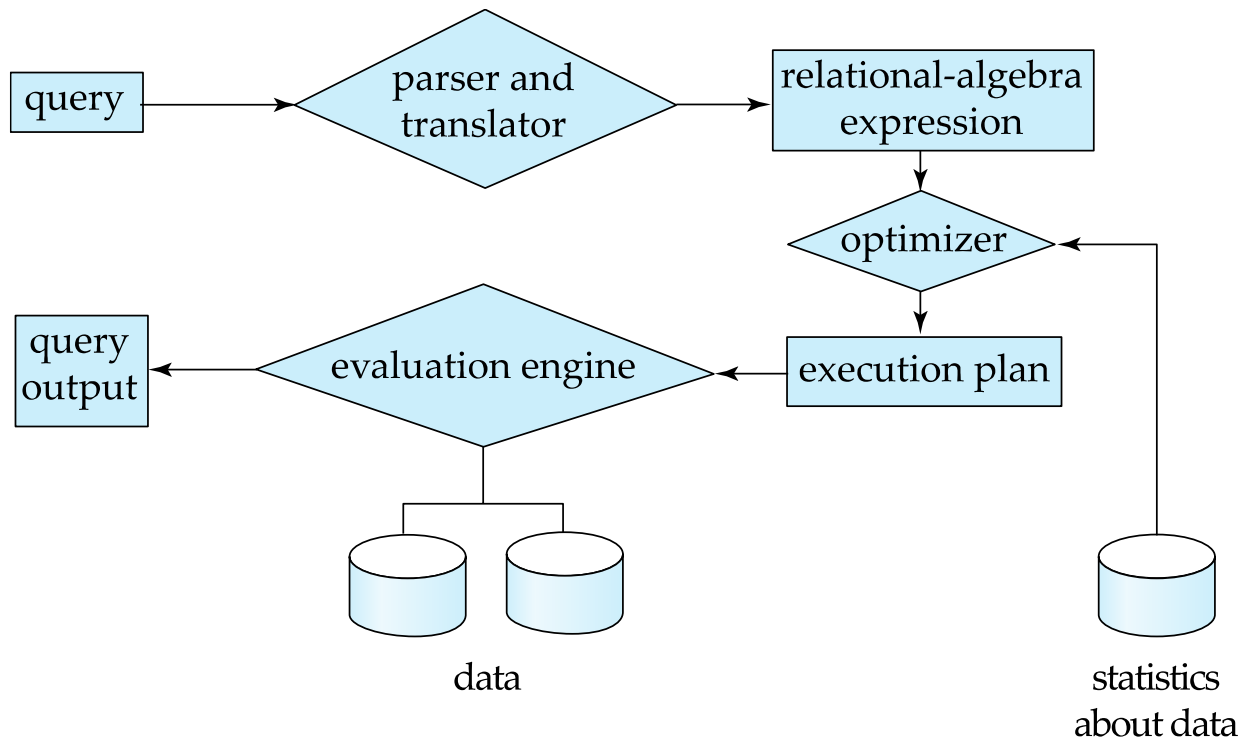


# What are we going to learn?

- How efficiently the queries can be answered?
  - Query processing
    - query translation, query optimizing transformations, query evaluation
  - Query Optimization
    - find the lowest cost plan of evaluating a given query

# Basic Steps in Query Processing

1. Parsing and translation
2. Optimization
3. Evaluation





# Basic Steps in Query Processing (Cont.)

- **Parsing and translation**
  - translate the query into its internal form. This is then translated into relational algebra.
  - Parser checks syntax and semantics (verifies relations etc.)
- **Evaluation**
  - The query-execution engine takes a query-evaluation plan, executes that plan, and returns the answers to the query.



# Basic Steps in Query Processing: Optimization

- A relational algebra expression may have many equivalent expressions
  - E.g.,  $\sigma_{salary < 75000}(\Pi_{salary}(instructor))$  is equivalent to  $\Pi_{salary}(\sigma_{salary < 75000}(instructor))$
- Each relational algebra operation can be evaluated using one of several different algorithms
  - Correspondingly, a relational-algebra expression can be evaluated in many ways.
- Annotated expression specifying detailed evaluation strategy is called an **evaluation-plan**. E.g.,:
  - Use an index on *salary* to find instructors with salary < 75000,
  - Or perform complete relation scan and discard instructors with salary  $\geq 75000$



# Basic Steps: Optimization (Cont.)

- **Query Optimization:** Amongst all equivalent evaluation plans choose the one with lowest cost.
  - Cost is estimated using statistical information from the database catalog
    - e.g.. number of tuples in each relation, size of tuples, etc.
- Key issues
  - How to measure query costs
  - Algorithms for evaluating relational algebra operations
  - How to combine algorithms for individual operations in order to evaluate a complete expression
  - How to optimize queries, that is, how to find an evaluation plan with lowest estimated cost





# Measures of Query Cost

- Many factors contribute to time cost
  - *disk access, CPU, and network communication*
- Cost can be measured based on
  - **response time**, i.e. total elapsed time for answering query, or
  - total **resource consumption**
- We use total resource consumption as cost metric
  - Response time harder to estimate, and minimizing resource consumption is a good idea in a shared database
- We ignore CPU costs for simplicity
  - Real systems do take CPU cost into account
  - Network costs must be considered for parallel systems
- We describe how to estimate the cost of each operation
  - We do not include cost to writing output to disk



# Measures of Query Cost

- For simplicity we use the **number of block transfers from disk and the number of seeks** (I/O access) as the cost measures
  - $t_T$  – time to transfer one block
    - Assuming for simplicity that write cost is same as read cost
  - $t_S$  – time for one seek
  - Cost for  $b$  block transfers plus  $S$  seeks
$$b * t_T + S * t_S$$
- $t_S$  and  $t_T$  depend on where data is stored; with 4 KB blocks:
  - High end magnetic disk:  $t_S = 4$  msec and  $t_T = 0.1$  msec
  - SSD:  $t_S = 20$ -90 microsec and  $t_T = 2$ -10 microsec for 4KB



# Measures of Query Cost (Cont.)

- Required data may be buffer resident already, avoiding disk I/O
  - But hard to take into account for cost estimation
- Several algorithms can reduce disk IO by using extra buffer space
  - Amount of real memory available to buffer depends on other concurrent queries and OS processes, known only during execution
- Worst case estimates assume that no data is initially in buffer and only the minimum amount of memory needed for the operation is available
  - But more optimistic estimates are used in practice



# Selection Operation

- File scan
- Algorithm **A1 (linear search)**. Scan each file block and test all records to see whether they satisfy the selection condition.
  - Cost estimate =  $b_r$  block transfers + 1 seek
    - $b_r$  denotes number of blocks containing records from relation  $r$
  - If selection is on a key attribute, can stop on finding record
    - cost =  $(b_r/2)$  block transfers + 1 seek
  - Linear search can be applied regardless of
    - selection condition or
    - ordering of records in the file, or
    - availability of indices



# Selections Using Indices

- **Index scan** – search algorithms that use an index
  - selection condition must be on search-key of index.
- **A2 (clustering index, equality on key)**. Retrieve a single record that satisfies the corresponding equality condition
  - $Cost = (h_i + 1) * (t_T + t_S)$
- **A3 (clustering index, equality on nonkey)**  
Retrieve multiple records.
  - Records will be on consecutive blocks
    - Let  $b$  = number of blocks containing matching records
  - $Cost = h_i * (t_T + t_S) + t_S + t_T * b$



# Selections Using Indices

- **A4 (secondary index, equality on key/non-key).**
  - Retrieve a single record if the search-key is a candidate key
    - $Cost = (h_i + 1) * (t_T + t_S)$
  - Retrieve multiple records if search-key is not a candidate key
    - each of  $n$  matching records may be on a different block
    - $Cost = (h_i + n) * (t_T + t_S)$ 
      - Can be very expensive!



# Selections Involving Comparisons

- Can implement selections of the form  $\sigma_{A \leq v}(r)$  or  $\sigma_{A \geq v}(r)$  by using
  - a linear file scan,
  - or by using indices in the following ways:
- **A5 (clustering index, comparison).** (Relation is sorted on A)
  - For  $\sigma_{A \geq v}(r)$  use index to find first tuple  $\geq v$  and scan relation sequentially from there
  - For  $\sigma_{A \leq v}(r)$  just scan relation sequentially till first tuple  $> v$ ; do not use index
- **A6 (secondary index, comparison).**
  - For  $\sigma_{A \geq v}(r)$  use index to find first index entry  $\geq v$  and scan index sequentially from there, to find pointers to records.
  - For  $\sigma_{A \leq v}(r)$  just scan leaf pages of index finding pointers to records, till first entry  $> v$
  - In either case, retrieve records that are pointed to
  - requires an I/O per record;



# Implementation of Complex Selections

- **Conjunction:**  $\sigma_{\theta_1 \wedge \theta_2 \wedge \dots \wedge \theta_n}(r)$
- **A7 (conjunctive selection using one index).**
  - Select a combination of  $\theta_i$  and algorithms A2 through A6 that results in the least cost for  $\sigma_{\theta_i}(r)$ .
  - Test other conditions on tuple after fetching it into memory buffer.
- **A8 (conjunctive selection using composite index).**
  - Use appropriate composite (multiple-key) index if available.
- **A9 (conjunctive selection by intersection of identifiers).**
  - Requires indices with record pointers.
  - Use corresponding index for each condition, and take intersection of all the obtained sets of record pointers.
  - Then fetch records from file
  - If some conditions do not have appropriate indices, apply test in memory.





# Algorithms for Complex Selections

- **Disjunction:**  $\sigma_{\theta_1 \vee \theta_2 \vee \dots \vee \theta_n}(r)$ .
- **A10 (disjunctive selection by union of identifiers).**
  - Applicable if *all* conditions have available indices.
    - Otherwise use linear scan.
  - Use corresponding index for each condition, and take union of all the obtained sets of record pointers.
  - Then fetch records from file
- **Negation:**  $\sigma_{\neg\theta}(r)$ 
  - Use linear scan on file
  - If very few records satisfy  $\neg\theta$ , and an index is applicable to  $\theta$ 
    - Find satisfying records using index and fetch from file



# Join Operation

- Several different algorithms to implement joins
  - Nested-loop join
  - Block nested-loop join
  - Indexed nested-loop join
- Choice based on cost estimate
- Examples use the following information
  - Number of records of *student*: 5,000    *takes*: 10,000
  - Number of blocks of *student*: 100    *takes*: 400



# Nested-Loop Join

- To compute the theta join  $r \bowtie_{\theta} s$
- - for each** tuple  $t_r$  **in**  $r$  **do begin**
  - for each** tuple  $t_s$  **in**  $s$  **do begin**
  - test pair  $(t_r, t_s)$  to see if they satisfy the join condition  $\theta$
  - if they do, add  $t_r \bullet t_s$  to the result.
  - end**
  - end**
- $r$  is called the **outer relation** and  $s$  the **inner relation** of the join.
- Requires no indices and can be used with any kind of join condition.
- Expensive since it examines every pair of tuples in the two relations.



# Nested-Loop Join (Cont.)

- In the worst case, if there is enough memory only to hold one block of each relation, the estimated cost is  $n_r * b_s + b_r$  block transfers, plus  $n_r + b_r$  seeks
- If the smaller relation fits entirely in memory, use that as the inner relation.
  - Reduces cost to  $b_r + b_s$  block transfers and 2 seeks
- Assuming worst case memory availability cost estimate is
  - with *student* as outer relation:
    - $5000 * 400 + 100 = 2,000,100$  block transfers,
    - $5000 + 100 = 5100$  seeks
  - with *takes* as the outer relation
    - $10000 * 100 + 400 = 1,000,400$  block transfers and 10,400 seeks
- If smaller relation (*student*) fits entirely in memory, the cost estimate will be 500 block transfers.
- Block nested-loops algorithm (next slide) is preferable.



# Block Nested-Loop Join

- Variant of nested-loop join in which every block of inner relation is paired with every block of outer relation.

```
for each block  $B_r$  of  $r$  do begin  
  for each block  $B_s$  of  $s$  do begin  
    for each tuple  $t_r$  in  $B_r$  do begin  
      for each tuple  $t_s$  in  $B_s$  do begin  
        Check if  $(t_r, t_s)$  satisfy the join condition  
        if they do, add  $t_r \bullet t_s$  to the result.  
      end  
    end  
  end  
end
```



# Block Nested-Loop Join (Cont.)

- Worst case estimate:  $b_r * b_s + b_r$  block transfers + 2 \*  $b_r$  seeks
  - Each block in the inner relation  $s$  is read once for each *block* in the outer relation
- Best case:  $b_r + b_s$  block transfers + 2 seeks.
- Improvements to nested loop and block nested loop algorithms:
  - In block nested-loop, use  $M - 2$  disk blocks as blocking unit for outer relations, where  $M$  = memory size in blocks; use remaining two blocks to buffer inner relation and output
    - Cost =  $\left\lceil \frac{b_r}{M-2} \right\rceil * b_s + b_r$  block transfers +  $2 \left\lceil \frac{b_r}{M-2} \right\rceil$  seeks
  - If equi-join attribute forms a key on inner relation, stop inner loop on first match
  - Scan inner loop forward and backward alternately, to make use of the blocks remaining in buffer
  - Use index on inner relation if available (next slide)



# Indexed Nested-Loop Join

- Index lookups can replace file scans if
  - join is an equi-join or natural join and
  - an index is available on the inner relation's join attribute
    - Can construct an index just to compute a join.
- For each tuple  $t_r$  in the outer relation  $r$ , use the index to look up tuples in  $s$  that satisfy the join condition with tuple  $t_r$
- Worst case: buffer has space for only one block of  $r$ , and, for each tuple in  $r$ , we perform an index lookup on  $s$ .
- Cost of the join:  $b_r (t_T + t_S) + n_r * c$ 
  - Where  $c$  is the cost of traversing index and fetching all matching  $s$  tuples for one tuple of  $r$
  - $c$  can be estimated as cost of a single selection on  $s$  using the join condition.
- If indices are available on join attributes of both  $r$  and  $s$ , use the relation with fewer tuples as the outer relation.



# Example of Nested-Loop Join Costs

- Compute *student* ⋈ *takes*, with *student* as the outer relation.
- Let *takes* have a primary B<sup>+</sup>-tree index on the attribute *ID*, which contains 20 entries in each index node.
- Since *takes* has 10,000 tuples, the height of the tree is 4, and one more access is needed to find the actual data
- *student* has 5000 tuples
- Cost of block nested loops join
  - $400 * 100 + 100 = 40,100$  block transfers +  $2 * 100 = 200$  seeks
    - assuming worst case memory
    - may be significantly less with more memory
- Cost of indexed nested loops join
  - $100 + 5000 * 5 = 25,100$  block transfers and seeks.
  - CPU cost likely to be less than that for block nested loops join





# Evaluation of Expressions

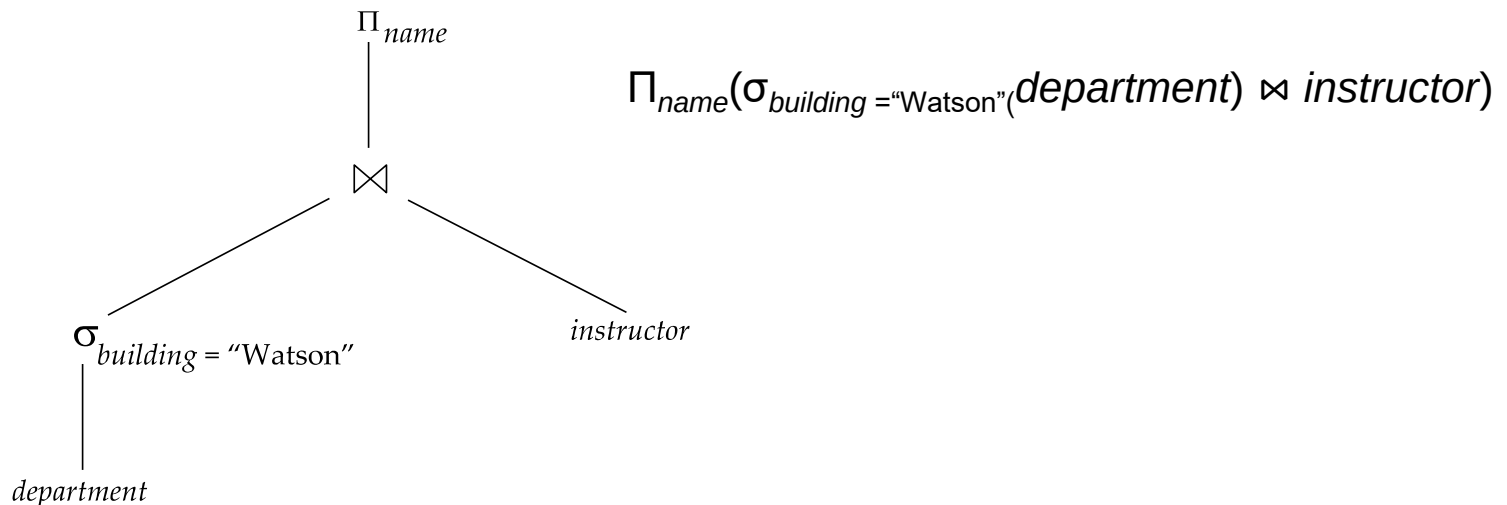
- So far: we have seen algorithms for individual operations
- Alternatives for evaluating an entire expression tree
  - **Materialization:** generate results of an expression whose inputs are relations or are already computed, **materialize** (store) it on disk. Repeat.
  - **Pipelining:** pass on tuples to parent operations even as an operation is being executed
- We study above alternatives in more detail

# Materialization

- **Materialized evaluation:** evaluate one operation at a time, starting at the lowest-level. Use intermediate results materialized into temporary relations to evaluate next-level operations.
- E.g., in figure below, compute and store

$$\sigma_{\text{building}=\text{"Watson"}}(\textit{department})$$

then compute and store its join with *instructor*, and finally compute the projection on *name*.





# Materialization (Cont.)

- Materialized evaluation is always applicable
- Cost of writing results to disk and reading them back can be quite high
  - Our cost formulas for operations ignore cost of writing results to disk, so
    - Overall cost = Sum of costs of individual operations + cost of writing intermediate results to disk
- **Double buffering**: use two output buffers for each operation, when one is full write it to disk while the other is getting filled
  - Allows overlap of disk writes with computation and reduces execution time



# Pipelining

- **Pipelined evaluation:** evaluate several operations simultaneously, passing the results of one operation on to the next.
- E.g., in previous expression tree, don't store result of

$$\sigma_{building="Watson"}(department)$$

- instead, pass tuples directly to the join. Similarly, don't store result of join, pass tuples directly to projection.
- Much cheaper than materialization: no need to store a temporary relation to disk.
- Pipelining may not always be possible – e.g., sort, hash-join.
- For pipelining to be effective, use evaluation algorithms that generate output tuples even as tuples are received for inputs to the operation.



# Pipelining (Cont.)

- Pipelines can be executed in two ways: **demand driven** and **producer driven**
- In **demand driven** or **lazy** evaluation
  - system repeatedly requests next tuple from the operation at the top level
  - Each operation requests next tuple from children operations as required, in order to output its next tuple
  - In between calls, operation has to maintain “**state**” so it knows what to return next
- In **producer-driven** or **eager** pipelining
  - Operators produce tuples eagerly and pass them up to their parents
    - Buffer maintained between operators, child puts tuples in buffer, parent removes tuples from buffer
    - if buffer is full, child waits till there is space in the buffer, and then generates more tuples
  - Each operation is modeled as a separate process or thread within the system
- Alternative name: **pull** and **push** models of pipelining



# Pipelining (Cont.)

- Implementation of demand-driven pipelining
  - Each operation is implemented as an **iterator** implementing the following operations
    - **open()**
      - E.g., file scan: initialize file scan
        - » state: pointer to beginning of file
      - E.g., merge join: sort relations;
        - » state: pointers to beginning of sorted relations
    - **next()**
      - E.g., for file scan: Output next tuple, and advance and store file pointer
      - E.g., for merge join: continue with merge from earlier state till next output tuple is found. Save pointers as iterator state.
    - **close()**



# Evaluation Algorithms for Pipelining

- Query plans are annotated to mark edges
  - pipelined edges
  - blocking edges (or materialized edges)
- Query plan can be divided into subtrees (**pipeline stage**)
  - each subtree has only pipelined edges
  - edges between the subtrees are non-pipelined
- Query processor executes the plan
  - one pipeline stage at a time
  - concurrently executes all operators in a single pipeline stage
- **Blocking operations**: cannot generate any output until all input is consumed
  - E.g., sorting, aggregation, ...



# Problem

- R1 relation : 20,000 tuples, 25 tuples in one block
- R2 relation: 45,000 tuples, 30 tuples in one block
- Estimate the number of block transfers and seeks required using each of the following join strategies:
  - Nested-loop join
  - Block nested-loop join
  - Merge join
  - Hash join





# Summary

- Query processing : parsing and translation, optimization, and execution.
- query evaluation cost is measured in terms of : number of blocks transfer and number of seeks.
- algorithms to execute select operation can broadly be categorized into: file scan and index scan
- There are several algorithms for computing the join of relations
  - Nested-loop join
    - Expensive in terms of number of block transfers and number of seeks
    - Do not require indices
  - Block nested –loop join
    - Less expensive than nested loop, various strategies can be applied to reduce the cost further
  - Indexed nested-loop
    - Index look-up replaces file scans for inner relation
- Basics of expression evaluation using materialization and pipelining